CHAPTER 1

Deep learning foundations for NLP

In Chapter **??**, we formalized language modeling through the exact decomposition of the chain rule of probability. However, empirical evaluation demonstrates the fundamental structural limitations of discrete counting models: table-based frequency estimators suffer from exponential parameter explosion (V^n), context horizon amnesia ($n-1$), and total semantic blindness across distinct vocabulary symbols. Because discrete strings cannot be differentiated and isolated token IDs share zero statistical strength, classical frequency tables cannot scale to the combinatorial complexity of human language or software engineering source code.

To transcend the discrete counting paradigm, modern computational linguistics and artificial intelligence reframe language processing through the lens of *connectionism* and differentiable programming. Instead of tabulating occurrences in discrete bins, we parameterize language predictors as continuous mathematical functions mapping inputs onto high-dimensional metric spaces $\mathbb{R}^d$. In this continuous formulation, semantic similarity corresponds to geometric proximity, syntactic constraints correspond to smooth decision manifolds, and model parameters are learned automatically by propagating error gradients through computational graphs via multivariable calculus.

This chapter establishes the rigorous mathematical and architectural foundations of deep learning required to build modern foundation models. Beginning with the biological inspiration and mathematical formulation of the artificial perceptron, we prove why single layers fail on non-linear tasks (such as XOR) and why multi-layer linear stacking without activations collapses into a single trivial matrix. We then analyze the functional geometry of non-linear activations ($\sigma, \tanh, \text{ReLU}, \text{GELU}$), formalize the Universal Approximation Theorem through localized wavelet bump constructions, derive the convexity of maximum likelihood loss landscapes versus Mean Squared Error, and unpack the exact mechanics of reverse-mode automatic differentiation across directed acyclic graphs.

1.1 From biological neurons to the artificial perceptron

Having established the probabilistic formulation of language modeling and the combinatorial limitations of discrete symbol counting in Chapter **??**, we now develop the neural computational foundations that allow language models to scale. In contemporary Natural Language Processing and AI-assisted Software Engineering, next-token prediction is reformulated as a high-dimensional, non-linear classification problem over continuous vector spaces. Rather than storing astronomical tables of discrete counts, we parameterize language predictors as continuous functions whose weights are optimized via gradient descent.

The foundational building block of modern deep learning is the *perceptron*, originally inspired by biological neurons in the human nervous system. A biological neuron receives electrochemical signals from upstream cells through branching *dendrites*, integrates these incoming potentials in its central cell body (*soma*), and, when the accumulated electrical voltage exceeds a physiological threshold (the action potential), fires a spike through its *axon* to downstream synapses.

In computational neural networks, this biological mechanism was first formalized in 1943 [**?**] as a threshold logic unit and subsequently refined in 1958 [**?**] into the trainable *Perceptron*.

Definition 1.1: *The artificial perceptron*

Let $\mathbf{x} = [x_1, x_2, \ldots, x_n]^\top \in \mathbb{R}^n$ represent an input feature vector, and let $\mathbf{w} = [w_1, w_2, \ldots, w_n]^\top \in \mathbb{R}^n$ represent the corresponding parameter weights, with scalar bias $w_0 = b \in \mathbb{R}$. The artificial perceptron computes:

$$y = f\left(w_0 + \sum_{i=1}^{n} w_i x_i\right) = f\left(w_0 + w_1 x_1 + w_2 x_2 + \cdots + w_n x_n\right), \tag{1.1}$$

where $f : \mathbb{R} \to \mathbb{R}$ is the activation function. By adopting the standard convention of augmenting the input vector with a constant dummy feature $x_0 = 1$ and defining the parameter vector as $\mathbf{w} = [w_0, w_1, \ldots, w_n]^\top \in \mathbb{R}^{n+1}$, Equation (1.1) is expressed in compact vector inner-product notation:

$$y = f\left(\sum_{i=0}^{n} w_i x_i\right) = f\left(\mathbf{w}^\top \mathbf{x}\right) \quad \text{with } x_0 = 1,\ w_0 = \text{bias}. \tag{1.2}$$

Figure 1.1 illustrates the structural mechanics of the artificial perceptron: input signals are weighted by synaptic connection strengths, summed at a central junction, and passed through an activation function to generate the output signal.

1.1.1 The perceptron in two dimensions: linear decision hyperplane

To establish geometric intuition, consider a two-dimensional feature space $\mathbf{x} = [x_1, x_2]^\top \in \mathbb{R}^2$ with weights $\mathbf{w} = [w_1, w_2]^\top$ and bias w_0. The pre-activation value is:

$$z(\mathbf{x}) = w_1 x_1 + w_2 x_2 + w_0. \tag{1.3}$$

When the perceptron acts as a binary classifier using a step or sign activation ($y = \text{sgn}(z)$), the decision boundary separating the two output classes is precisely the set of points where the pre-activation vanishes:

$$\mathcal{H} = \left\{\mathbf{x} \in \mathbb{R}^2 \mid w_1 x_1 + w_2 x_2 + w_0 = 0\right\}. \tag{1.4}$$

Geometrically, Equation (1.4) defines a straight line in the (x_1, x_2) Cartesian plane governed by two primary properties:

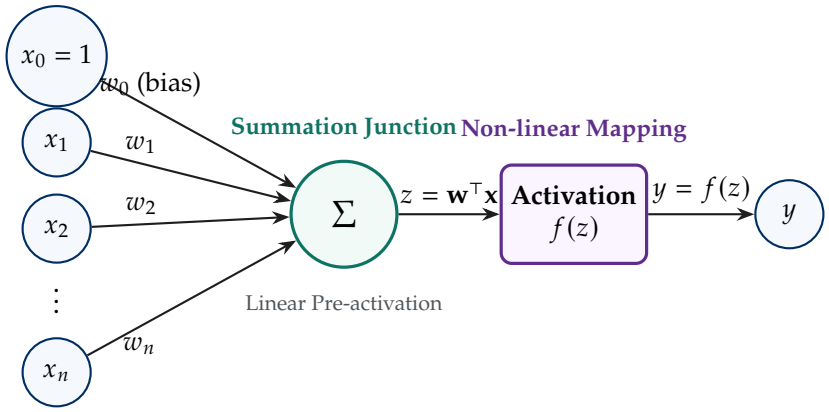


Figure 1.1: Architectural schematic of the artificial perceptron. Inputs x_i are modulated by synaptic weights w_i, accumulated with bias w_0 at the summation junction Σ to yield pre-activation $z = \mathbf{w}^\top \mathbf{x}$, and mapped through activation function $f(z)$ to produce scalar output y.

- **Normal Vector**: The weight vector $\mathbf{w} = [w_1, w_2]^\top$ is strictly perpendicular (orthogonal) to the separating line $\mathcal{H}$. For any two points $\mathbf{x}_a, \mathbf{x}_b \in \mathcal{H}$:

$$\mathbf{w}^\top \mathbf{x}_a + w_0 = 0 \quad \text{and} \quad \mathbf{w}^\top \mathbf{x}_b + w_0 = 0 \implies \mathbf{w}^\top (\mathbf{x}_a - \mathbf{x}_b) = 0.$$

  The vector $\mathbf{w}$ points directly into the positive decision region ($z > 0$).

- **Orthogonal Distance to Origin**: The shortest Euclidean distance from the origin $\mathbf{0} = [0, 0]^\top$ to the separating hyperplane $\mathcal{H}$ is:

$$d = \frac{|w_0|}{\|\mathbf{w}\|_2} = \frac{|w_0|}{\sqrt{w_1^2 + w_2^2}}. \tag{1.5}$$

  The bias w_0 shifts the hyperplane parallel to itself: if $w_0 = 0$, the decision boundary passes exactly through the origin; adjusting w_0 translates the boundary without altering its orientation.

Figure 1.2 diagrams this geometric relationship in $\mathbb{R}^2$.

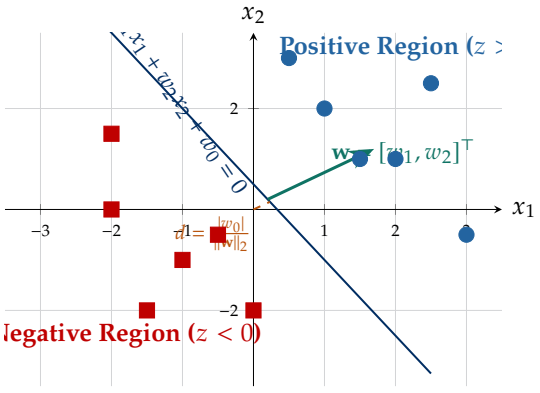


Figure 1.2: Geometric representation of the perceptron decision boundary in $\mathbb{R}^2$. The line $w_1 x_1 + w_2 x_2 + w_0 = 0$ partitions feature space into positive ($\mathcal{H}^+$) and negative ($\mathcal{H}^-$) half-spaces. The weight vector $\mathbf{w}$ defines the normal vector to the separating hyperplane, and the scalar bias w_0 determines its distance from the origin.

1.1.2 Connection to classical linear and logistic regression

The artificial perceptron unifies two foundational statistical models studied in classical data analysis:

- **Linear Regression**: If the activation function is the linear identity $f(z) = z$, Equation (1.2) computes:
$$y = \mathbf{w}^\top \mathbf{x},$$
predicting an unbounded real-valued continuous response variable.
- **Logistic Regression**: If the activation function is the logistic sigmoid $f(z) = \sigma(z) = \frac{1}{1+e^{-z}}$, Equation (1.2) computes:
$$y = \sigma(\mathbf{w}^\top \mathbf{x}) = \mathbb{P}(Y = 1 \mid \mathbf{x}),$$
predicting the posterior Bernoulli probability of belonging to the positive class.

1.1.3 Linear inseparability and the XOR limitation of single-layer perceptrons

Despite its mathematical elegance, the single-layer perceptron encounters a strict representational ceiling on non-linear problems. In their seminal 1969 monograph *"Perceptrons"*, Marvin Minsky and Seymour Papert demonstrated that a single-layer perceptron cannot compute the basic *exclusive-or* (XOR) logical function.

Consider the XOR logic function over binary inputs $x_1, x_2 \in \{0, 1\}$:

x_1	x_2	Target $y = x_1 \oplus x_2$
0	0	0
0	1	1
1	0	1
1	1	0

For a single-layer perceptron with weights w_1, w_2 and bias w_0 to compute this function, there must exist parameters such that the linear boundary $w_1x_1 + w_2x_2 + w_0$ outputs positive values ($z \geq 0$) for target 1 and negative values ($z < 0$) for target 0. Substituting the four input pairs yields the system of linear inequalities:

$$(x_1 = 0, x_2 = 0) \implies w_0 < 0, \tag{1.6}$$
$$(x_1 = 1, x_2 = 0) \implies w_1 + w_0 \geq 0, \tag{1.7}$$
$$(x_1 = 0, x_2 = 1) \implies w_2 + w_0 \geq 0, \tag{1.8}$$
$$(x_1 = 1, x_2 = 1) \implies w_1 + w_2 + w_0 < 0. \tag{1.9}$$

We formalize this impossibility through a direct proof by contradiction:
Summing inequalities (1.7) and (1.8):

$$(w_1 + w_0) + (w_2 + w_0) \geq 0 \implies w_1 + w_2 + 2w_0 \geq 0 \implies w_1 + w_2 \geq -2w_0. \tag{1.10}$$

Now examine inequality (1.9):

$$w_1 + w_2 + w_0 < 0 \implies w_1 + w_2 < -w_0. \tag{1.11}$$

Combining inequalities (1.10) and (1.11) yields:

$$-2w_0 \leq w_1 + w_2 < -w_0 \implies -2w_0 < -w_0.$$

Adding $2w_0$ to both sides yields:

$$0 < w_0 \implies w_0 > 0.$$

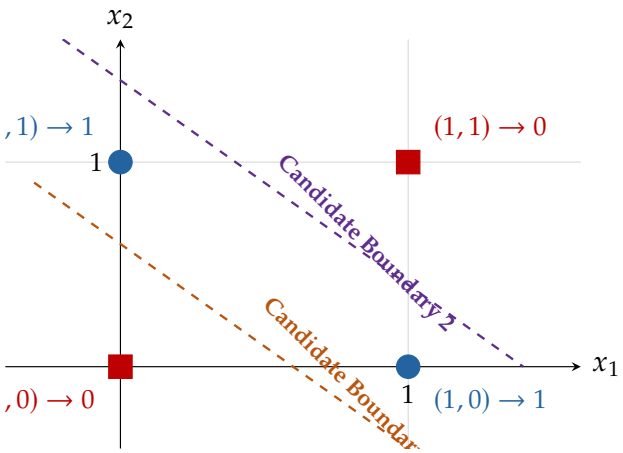


Figure 1.3: The XOR linear inseparability barrier (Minsky & Papert 1969). Class 1 points $(0, 1)$ and $(1, 0)$ (blue circles) cannot be separated from Class 0 points $(0, 0)$ and $(1, 1)$ (red squares) by any single linear hyperplane $w_1 x_1 + w_2 x_2 + w_0 = 0$. Any single line either isolates $(0, 0)$ or $(1, 1)$, but cannot simultaneously group both off-diagonal points.

However, inequality (1.6) strictly demands that $w_0 < 0$. The assumption that a single linear hyperplane can compute XOR produces a direct mathematical contradiction ($w_0 > 0$ and $w_0 < 0$). ■

As visualized in Figure 1.3, no straight line can separate the diagonal pairs. Because natural language and software engineering phenomena are riddled with non-linear relationships (e.g., mutually dependent programming idioms, nested conditional scopes), single-layer linear perceptrons are fundamentally inadequate.

This fundamental limitation raises an immediate architectural question: can non-linear decision boundaries be constructed simply by cascading multiple linear perceptrons into a multi-layer network without non-linear activations? We evaluate this hypothesis analytically in the next section.

1.2 Stacking linear layers and the proof of linear stacking collapse

1.2.1 Fully connected linear layers (dense layers)

In multi-layer architectures, individual perceptrons do not operate in isolation; instead, multiple neurons run concurrently across a shared input vector $\mathbf{x} \in \mathbb{R}^{d_{\text{in}}}$, forming a *dense layer* (or fully connected layer).

Let d_{in} denote the input feature dimensionality and d_{out} denote the number of hidden or output units. Stacking the individual weight vectors $\mathbf{w}_j \in \mathbb{R}^{d_{\text{in}}}$ as columns of a weight matrix $\mathbf{W} \in \mathbb{R}^{d_{\text{in}} \times d_{\text{out}}}$ and collecting the scalar biases b_j into a bias vector $\mathbf{b} \in \mathbb{R}^{d_{\text{out}}}$, the layer computation expresses compactly in matrix notation:

$$\mathbf{y} = f\left(\mathbf{W}^\top \mathbf{x} + \mathbf{b}\right), \tag{1.12}$$

where $f(\cdot)$ is applied elementwise. Figure 1.4 diagrams this fully connected bipartite structure with explicit tensor dimensions and connection weights.

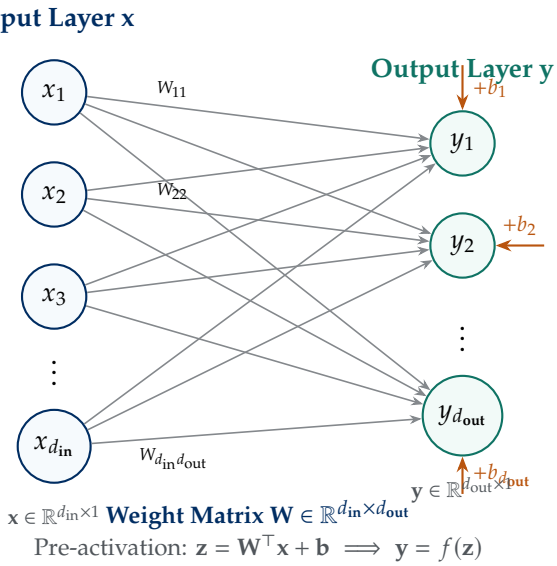


Figure 1.4: Architecture of a fully connected dense layer mapping an input vector $\mathbf{x} \in \mathbb{R}^{d_{\text{in}}}$ to output $\mathbf{y} \in \mathbb{R}^{d_{\text{out}}}$. Each directed edge carries an individual scalar weight W_{ij}, and each output neuron incorporates an additive bias b_j.

For an affine transformation mapping $\mathbb{R}^{d_{\text{in}}} \to \mathbb{R}^{d_{\text{out}}}$:

$$N_{\text{params}} = \underbrace{d_{\text{in}} \times d_{\text{out}}}_{\text{weight parameters}} + \underbrace{d_{\text{out}}}_{\text{bias parameters}} = d_{\text{out}}(d_{\text{in}} + 1). \tag{1.13}$$

Example Calculation: Consider a two-layer Multi-Layer Perceptron (MLP) mapping an embedding of dimension $d_{\text{in}} = 768$ through a hidden layer $d_{\text{hidden}} = 3072$ to an output projection $d_{\text{out}} = 768$:

1. Layer 1: $3072 \times (768 + 1) = 2{,}359{,}296 + 3072 = 2{,}362{,}368$ parameters.
2. Layer 2: $768 \times (3072 + 1) = 2{,}359{,}296 + 768 = 2{,}360{,}064$ parameters.
3. Total Parameters: $2{,}362{,}368 + 2{,}360{,}064 = 4{,}722{,}432$ parameters.

It is essential to account for the additive bias parameters (d_{out}) alongside the multiplicative weight matrix ($d_{\text{in}} \times d_{\text{out}}$) to obtain exact parameter counts.

1.2.2 The mathematical proof of linear stacking collapse

Now we return to our hypothesis: *Can we solve XOR and non-linear problems simply by cascading multiple linear layers without activation functions?*

The following theorem demonstrates that without non-linear activations, stacking arbitrary numbers of linear layers provides zero additional representational power over a single-layer linear model.

Theorem 1.1: *Linear stacking collapse*

Let $f_1 : \mathbb{R}^n \to \mathbb{R}^m$ and $f_2 : \mathbb{R}^m \to \mathbb{R}^k$ denote two consecutive fully connected layers operating without non-linear activations ($f(x) = x$). Let $\mathbf{W}_1 \in \mathbb{R}^{n \times m}$ and $\mathbf{b}_1 \in \mathbb{R}^m$ be the parameters of the first layer, and let $\mathbf{W}_2 \in \mathbb{R}^{m \times k}$ and $\mathbf{b}_2 \in \mathbb{R}^k$ be the parameters of the second layer:

$$\mathbf{h} = f_1(\mathbf{x}) = \mathbf{W}_1^\top \mathbf{x} + \mathbf{b}_1, \tag{1.14}$$

$$\mathbf{z} = f_2(\mathbf{h}) = \mathbf{W}_2^\top \mathbf{h} + \mathbf{b}_2. \tag{1.15}$$

The composite function $\mathbf{z} = f_2(f_1(\mathbf{x}))$ collapses identically to a single affine transformation:

$$\begin{aligned} \mathbf{z} &= \mathbf{W}_2^\top \left(\mathbf{W}_1^\top \mathbf{x} + \mathbf{b}_1\right) + \mathbf{b}_2 \\ &= \mathbf{W}_2^\top \mathbf{W}_1^\top \mathbf{x} + \mathbf{W}_2^\top \mathbf{b}_1 + \mathbf{b}_2 \\ &= (\mathbf{W}_1 \mathbf{W}_2)^\top \mathbf{x} + \left(\mathbf{W}_2^\top \mathbf{b}_1 + \mathbf{b}_2\right) \\ &= \mathbf{W}_{\text{net}}^\top \mathbf{x} + \mathbf{b}_{\text{net}}, \end{aligned} \tag{1.16}$$

where $\mathbf{W}_{\text{net}} = \mathbf{W}_1 \mathbf{W}_2 \in \mathbb{R}^{n \times k}$ and $\mathbf{b}_{\text{net}} = \mathbf{W}_2^\top \mathbf{b}_1 + \mathbf{b}_2 \in \mathbb{R}^k$.

Matrix multiplication is associative and closed under composition. The product of two matrices $\mathbf{W}_1 \in \mathbb{R}^{n \times m}$ and $\mathbf{W}_2 \in \mathbb{R}^{m \times k}$ is simply a single matrix $\mathbf{W}_{\text{net}} \in \mathbb{R}^{n \times k}$. Likewise, the affine transformation of bias vector $\mathbf{b}_1$ by $\mathbf{W}_2^\top$ added to $\mathbf{b}_2$ is simply a single constant vector $\mathbf{b}_{\text{net}} \in \mathbb{R}^k$.

By mathematical induction, for any cascade of L linear layers:

$$\mathbf{z} = \mathbf{W}_L^\top \left(\dots \left(\mathbf{W}_2^\top \left(\mathbf{W}_1^\top \mathbf{x} + \mathbf{b}_1\right) + \mathbf{b}_2\right) \dots\right) + \mathbf{b}_L = \mathbf{W}_{\text{net}}^\top \mathbf{x} + \mathbf{b}_{\text{net}}, \tag{1.17}$$

where:

$$\mathbf{W}_{\text{net}} = \prod_{l=1}^{L} \mathbf{W}_l \in \mathbb{R}^{d_{\text{in}} \times d_{\text{out}}}, \quad \mathbf{b}_{\text{net}} = \mathbf{b}_L + \sum_{l=1}^{L-1} \left(\prod_{j=l+1}^{L} \mathbf{W}_j \right)^\top \mathbf{b}_l. \tag{1.18}$$

The composite mapping is strictly linear. Stacking 100 purely linear layers introduces millions of redundant matrix multiplications while learning nothing beyond standard linear regression. It strictly cannot compute XOR or any non-linear function. ■

This collapse establishes that **non-linear activation functions are strictly mandatory**. Without non-linearity between layers, deep networks collapse into trivial shallow affine mappings. Non-linear activations warp and fold coordinate space, transforming linearly inseparable data manifolds into representations where classes can be cleanly separated.

1.2.3 Universal approximation theorem and bump-function synthesis

Given that non-linear activation functions prevent linear collapse, what family of functions can a non-linear neural network represent? The theoretical foundation validating neural networks as general-purpose function approximators is the *Universal Approximation Theorem*.

Theorem 1.2: *Universal approximation theorem*

Let $\sigma : \mathbb{R} \to \mathbb{R}$ be any continuous, non-constant, bounded, and monotonically increasing activation function (e.g., sigmoid) [**?**]. Let $I_n \subset \mathbb{R}^n$ denote a compact (closed and bounded) subset of n-dimensional Euclidean space, and let $C(I_n)$ denote the space of continuous real-valued functions on I_n. Then, for any target continuous function $g \in C(I_n)$ and any arbitrary error tolerance $\epsilon > 0$, there exist a finite integer $M \in \mathbb{N}$, parameter weights $\mathbf{w}_j \in \mathbb{R}^n$, outer weights $\alpha_j \in \mathbb{R}$, and biases $b_j \in \mathbb{R}$ such that the single-hidden-layer feedforward network:

$$F(\mathbf{x}) = \sum_{j=1}^{M} \alpha_j \sigma \left(\mathbf{w}_j^\top \mathbf{x} + b_j \right) \tag{1.19}$$

satisfies uniform approximation error:

$$\sup_{\mathbf{x} \in I_n} \left| F(\mathbf{x}) - g(\mathbf{x}) \right| < \epsilon. \tag{1.20}$$

Originally proved in [**?**] for sigmoidal activations and generalized in [**?**] and [**?**] to arbitrary continuous non-polynomial activations, this theorem guarantees that a feedforward network with a single hidden layer has the mathematical capacity to approximate any continuous function (as discussed in the course theoretical foundations).

To understand why a single hidden layer can approximate any continuous curve, consider a one-dimensional target function $g(x) : [a, b] \to \mathbb{R}$. The approximation proceeds in two primary steps:

Constructing localized rectangular bump functions

Consider two sigmoid neurons with very high slope $w \gg 0$ located at spatial offsets x_1 and x_2 (with $x_1 < x_2$). As $w \to \infty$, the sigmoid $\sigma(w(x - x_1))$ converges to a Heaviside step function $H(x - x_1)$.

Subtracting the second sigmoid from the first produces a localized *bump function* (or rectangular wavelet):

$$B(x) = \sigma(w(x - x_1)) - \sigma(w(x - x_2)) \approx \begin{cases} 1 & \text{if } x_1 \leq x \leq x_2 \\ 0 & \text{otherwise} \end{cases}. \tag{1.21}$$

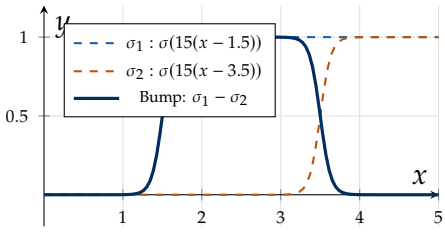
Figure 1.5(a) illustrates how two opposing sigmoids subtract to create a localized rectangular pulse.

Riemann sum function reconstruction

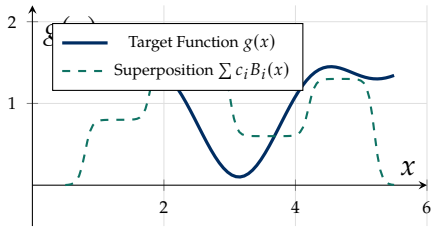
By the definition of continuous functions on compact intervals, $g(x)$ can be approximated to arbitrary accuracy ϵ by a piecewise-constant step function consisting of M uniform sub-intervals $[x_k, x_{k+1}]$ with heights $c_k = g(x_k)$, exactly as in standard Riemann integration:

$$g(x) \approx \sum_{k=1}^{M} c_k B_k(x) = \sum_{k=1}^{M} c_k \left[\sigma(w(x - x_k)) - \sigma(w(x - x_{k+1})) \right]. \tag{1.22}$$

Each bump $B_k(x)$ requires exactly two hidden sigmoid neurons. Consequently, a single hidden layer containing $2M$ neurons can synthesize any continuous function to arbitrary precision, as visualized in Figure 1.5(b).



(a) Bump Synthesis via Opposing Sigmoids



(b) Riemann-style Function Reconstruction

Figure 1.5: Visual derivation of the Universal Approximation Theorem. (a) Subtracting two shifted steep sigmoids produces a localized rectangular bump function $B(x)$; (b) Superimposing multiple scaled bump functions reconstructs any continuous target function to arbitrary accuracy ϵ.

1.2.4 Why deep networks if shallow networks are universal approximators?

If a single hidden layer can approximate any continuous function, why does modern deep learning exclusively build deep architectures with dozens or hundreds of stacked layers (e.g., 96 layers in GPT-3)?

Three foundational theoretical and geometric arguments answer this fundamental question:

1. **Non-Constructive Existence vs. Optimization**: The Cybenko and Hornik theorems guarantee that a set of parameters exists within some arbitrary ϵ-ball, but offer no gradient-based algorithm to discover them. In practice, gradient descent over shallow networks with massive width easily becomes trapped in degenerate saddle points or overfits catastrophic boundary noise.

2. **Exponential Parameter Inefficiency and the Curse of Dimensionality**: Approximating highly oscillatory or compositional functions in d_{in} dimensions with a single hidden layer requires a width M that scales exponentially with input dimensionality ($M \sim O(2^{d_{\text{in}}})$), rendering shallow networks computationally intractable for real-world NLP vocabularies and context windows.

3. **Exponential Linear Region Counting with Depth**: Simon Prince [**?**] and Montúfar et al. [**?**] mathematically formalized the expressive advantage of depth through *linear region counting*. A shallow network with 1 hidden layer containing K rectified linear units partitions input space $\mathbb{R}^{d_{\text{in}}}$ into at most:

$$\mathcal{N}_{\text{shallow}} \leq \sum_{j=0}^{d_{\text{in}}} \binom{K}{j} = O(K^{d_{\text{in}}}) \tag{1.23}$$

linear regions. In contrast, cascading D layers of width K allows subsequent layers to compute piecewise linear functions of piecewise linear functions, effectively folding the input domain repeatedly. The maximal number of linear regions generated by a deep ReLU network grows exponentially with depth D:

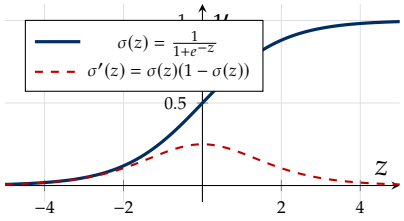
$$\mathcal{N}_{\text{deep}} = O\left(\left(\frac{K}{d_{\text{in}}}\right)^{(D-1)d_{\text{in}}} K^{d_{\text{in}}}\right) \sim O(K^D). \tag{1.24}$$

A deep network of depth D can thus produce an astronomical number of linear regions $O(K^D)$ using only $O(D \cdot K^2)$ total parameters. To match this expressive capacity, a shallow 1-layer network would require an exponential number of neurons $M \sim K^D$, necessitating astronomical parameter storage and memory bandwidth.

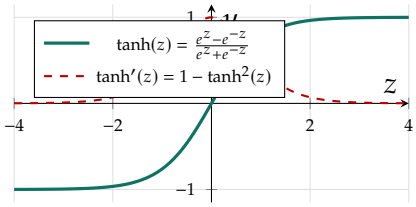
Deep architectures are therefore fundamentally more efficient: they factorize high-dimensional manifolds into hierarchical abstractions, extracting local lexical tokens at early layers, compositional syntactic trees at intermediate layers, and long-range semantic discourse at deep layers.

1.3 Taxonomy of neural activation functions

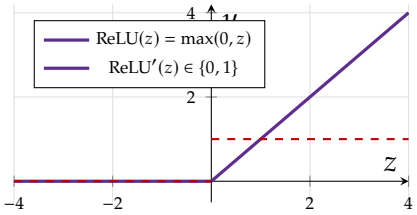
Modern neural NLP architectures deploy diverse activation functions tailored to hidden representations and output heads. Figure 1.6 presents exact mathematical plots and derivatives for the primary activation functions.



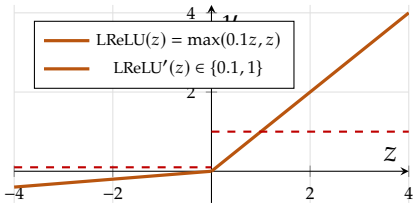
(a) Sigmoid (Logistic)



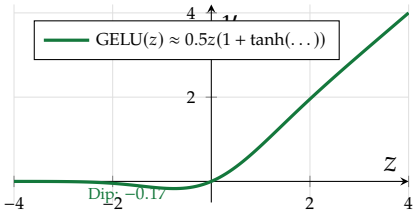
(b) Hyperbolic Tangent (tanh)



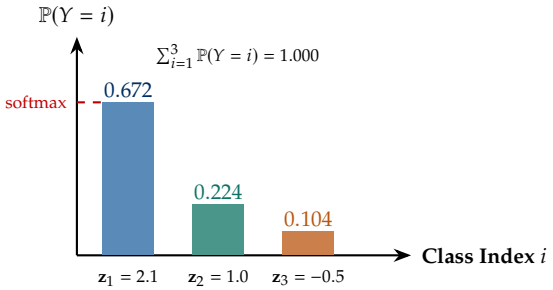
(c) Rectified Linear Unit (ReLU)



(d) Leaky ReLU ($\alpha = 0.1$)



(e) Gaussian Error Linear Unit (GELU)



(f) Softmax Simplex Mapping ($\mathbb{R}^3 \to \Delta^2$)

Figure 1.6: Taxonomy of primary neural activation functions plotted across their operating ranges: (a) Sigmoid $\sigma(z)$ with peak derivative 0.25; (b) Zero-centered $\tanh(z)$ with peak derivative 1.0; (c) Piecewise linear $\mathrm{ReLU}(z)$; (d) Leaky ReLU preventing dead units; (e) Non-monotonic $\mathrm{GELU}(z)$ used in modern Transformers; (f) Multi-class Softmax mapping logits to the probability simplex.

The logistic sigmoid

$$\sigma(z) = \frac{1}{1 + e^{-z}}, \quad \text{Range: } (0, 1), \quad \sigma'(z) = \sigma(z)(1 - \sigma(z)). \tag{1.25}$$

The derivative attains its global maximum of $\sigma'(0) = 0.25$ at the origin. For large positive or negative inputs ($|z| \geq 4$), $\sigma'(z) \to 0$, causing severe *vanishing gradients* in deep networks.

Hyperbolic tangent

$$\tanh(z) = \frac{e^{z} - e^{-z}}{e^{z} + e^{-z}} = 2\,\sigma(2z) - 1, \quad \text{Range: } (-1, 1), \quad \tanh'(z) = 1 - \tanh^2(z). \tag{1.26}$$

The hyperbolic tangent is strictly zero-centered, possessing average activation values near zero, which accelerates gradient descent compared to non-negative sigmoids. However, it still

saturates for $|z| \geq 3$, yielding vanishing gradients.

Rectified linear unit (ReLU)

$$\text{ReLU}(z) = \max(0, z), \quad \text{Range: } [0, \infty), \quad \text{ReLU}'(z) = \begin{cases} 1 & \text{if } z > 0 \\ 0 & \text{if } z < 0 \end{cases}. \tag{1.27}$$

ReLU completely eliminates gradient vanishing for positive pre-activations ($z > 0$). However, if a large gradient update shifts a neuron's weights such that $z \leq 0$ across all training samples, its activation and derivative permanently collapse to zero ($\text{ReLU}'(z) = 0$), a pathology known as the *dying ReLU problem*.

Leaky ReLU

$$\text{LReLU}(z) = \max(\alpha z, z) \quad \text{with } 0 < \alpha \ll 1, \quad \text{Range: } (-\infty, \infty), \quad \text{LReLU}'(z) = \begin{cases} 1 & \text{if } z > 0 \\ \alpha & \text{if } z \leq 0 \end{cases}. \tag{1.28}$$

By maintaining a small constant slope α (typically $\alpha = 0.01$) in the negative half-plane, Leaky ReLU guarantees that a non-zero gradient always backpropagates, preventing dead neurons.

Gaussian error linear unit (GELU)

Introduced by Dan Hendrycks and Kevin Gimpel, GELU scales the input by its cumulative standard normal probability $\Phi(z)$:

$$\text{GELU}(z) = z \cdot \Phi(z) = z \cdot \frac{1}{2}\left[1 + \text{erf}\left(\frac{z}{\sqrt{2}}\right)\right] \approx 0.5z\left(1 + \tanh\left(\sqrt{\frac{2}{\pi}}\left(z + 0.044715z^3\right)\right)\right). \tag{1.29}$$

GELU is smooth, differentiable everywhere, and non-monotonic, dipping to a global minimum of ≈ -0.17 at $z \approx -0.75$. This smoothness provides curvature information during optimization, making GELU the default activation in Transformer language models (GPT-2, GPT-3, BERT, RoBERTa).

Softmax normalization

For categorical multi-class predictions over C mutually exclusive classes, Softmax normalizes a real-valued logit vector $\mathbf{z} = [z_1, \ldots, z_C]^\top \in \mathbb{R}^C$ onto the unit probability simplex Δ^{C-1}:

$$\hat{y}_i = \text{softmax}(\mathbf{z})_i = \frac{e^{z_i}}{\sum_{j=1}^{C} e^{z_j}} \quad \text{such that } \hat{y}_i \in (0, 1) \quad \text{and} \quad \sum_{i=1}^{C} \hat{y}_i = 1. \tag{1.30}$$

Softmax forms the standard vocabulary output head of modern autoregressive language models, computing next-token probability distributions across vocabulary $\mathcal{V}$.

Table 1.1 provides a comparative reference summarizing the mathematical properties of each activation function.

Table 1.1: Comprehensive taxonomy of neural activation functions, mathematical formulations, operating bounds, and architectural domains.

Activation	Domain	Range	Peak Gradient	Primary Use Case in Language Modeling
Sigmoid (σ)	$\mathbb{R}$	$(0, 1)$	0.25 ($z = 0$)	Binary classification output heads; gating mechanisms in LSTM/GRU highways.
Tanh	$\mathbb{R}$	$(-1, 1)$	1.00 ($z = 0$)	Recurrent hidden state candidate generation; zero-centered coordinate representations.
ReLU	$\mathbb{R}$	$[0, \infty)$	1.00 ($z > 0$)	Classical deep feedforward networks; compute-efficient linear regions.
Leaky ReLU	$\mathbb{R}$	$\mathbb{R}$	1.00 ($z > 0$), α ($z \leq 0$)	Prevents dying neuron syndrome; robust discriminator networks.
GELU	$\mathbb{R}$	$[-0.17, \infty)$	≈ 1.08	Canonical Transformer activation (GPT, BERT); smooth, probabilistic gating.
Softmax	$\mathbb{R}^C$	Δ^{C-1}	Matrix Jacobian	Next-token prediction vocabulary head; maps logits to discrete probability simplex.
Identity	$\mathbb{R}$	$\mathbb{R}$	1.00 ($\forall z$)	Regression output head; unbounded linear prediction.

1.4 Architectural task formulations and software engineering case studies

In machine learning applications for software engineering, task formulations map specific software artifacts (commits, ASTs, bug reports, source code tokens) into continuous predictions. Three primary architectural heads dominate:

1.4.1 1. Binary classification: automated vulnerability detection

Consider predicting whether a proposed code commit contains a security vulnerability (e.g., buffer overflow, SQL injection).

- **Input Vector**: $\mathbf{x} \in \mathbb{R}^d$ (extracted code embeddings or static analysis metrics).
- **Output Layer**: A single linear neuron producing logit $z = \mathbf{w}^\top \mathbf{h} + b \in \mathbb{R}$.
- **Activation**: Logistic Sigmoid $\hat{y} = \sigma(z) = \mathbb{P}(\text{Vulnerable} = 1 \mid \mathbf{x}) \in [0, 1]$.
- **Decision Rule**: Flag for manual audit if $\hat{y} \geq \tau$ (where $\tau = 0.5$ by default, tuned to optimize recall).

Figure 1.7(a) illustrates the resulting separating hyperplane in feature space.

1.4.2 2. Multi-class classification: automated AST node type prediction

In program repair and neural compiler optimization, models classify Abstract Syntax Tree (AST) nodes into C discrete syntactic types (e.g., `MethodDeclaration`, `VariableDeclaration`, `IfStatement`, `WhileStatement`).

- **Output Layer**: C parallel linear neurons producing logit vector $\mathbf{z} = \mathbf{W}^\top \mathbf{h} + \mathbf{b} \in \mathbb{R}^C$.
- **Activation**: Softmax operator $\hat{\mathbf{y}} = \text{softmax}(\mathbf{z}) \in \Delta^{C-1}$.
- **Decision Rule**: Predicted AST type $\hat{c} = \arg\max_i \hat{y}_i$.

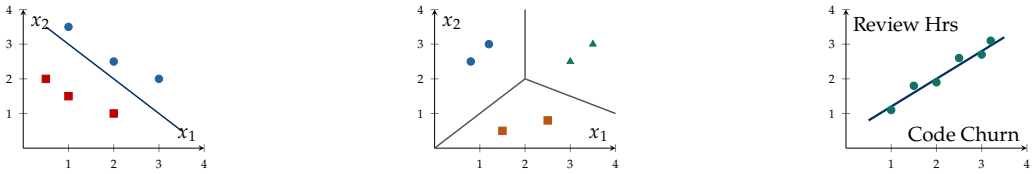
Figure 1.7(b) diagrams the resulting Voronoi-like partitioned multi-class decision space.

1.4.3 3. Regression: pull request review effort estimation

Engineering managers must predict continuous quantities, such as the estimated review time (in hours) required for an incoming pull request based on code churn, changed files, and author experience.

- **Output Layer**: A single linear neuron producing scalar $z = \mathbf{w}^\top \mathbf{h} + b$.
- **Activation**: Linear Identity function $f(z) = z \in \mathbb{R}$.
- **Prediction**: $\hat{y} = z \in \mathbb{R}$ (continuous unbounded prediction).

Figure 1.7(c) illustrates continuous line fitting through software engineering observation points.



(a) Binary: Defect Detection (b) Multi-class: AST Type (c) Regression: PR Review Effort

Figure 1.7: Task formulations across Software Engineering workflows: (a) Binary vulnerability detection with separating hyperplane; (b) Multi-class defect categorization with partitioned decision regions; (c) Continuous PR review effort regression with residual errors.

1.5 Loss function formulations and optimization landscapes

1.5.1 Loss function formulations

Training a neural network requires an objective function $\mathcal{L}(\theta)$ that measures the discrepancy between model predictions $\hat{y}$ and ground-truth targets y.

Mean squared error (MSE)

For continuous regression targets:

$$\mathcal{L}_{\text{MSE}}(\theta) = \frac{1}{N}\sum_{i=1}^{N}\left(y_i - \hat{y}_i\right)^2. \tag{1.31}$$

Binary cross-entropy (BCE)

For binary classification with target $y \in \{0,1\}$ and predicted probability $\hat{y} = \sigma(z) \in [0,1]$:

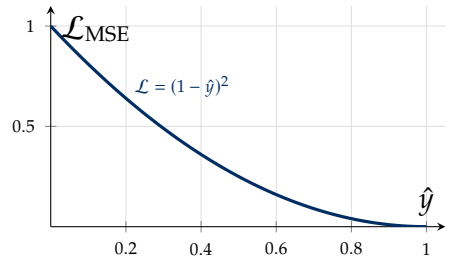
$$\mathcal{L}_{\text{BCE}}(\theta) = -y\ln(\hat{y}) - (1-y)\ln(1-\hat{y}). \tag{1.32}$$

Categorical cross-entropy (CCE)

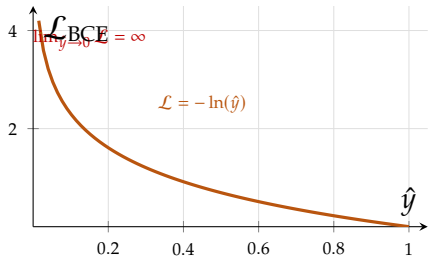
For multi-class classification over C mutually exclusive classes with one-hot target vector $\mathbf{y} \in \{0,1\}^C$:

$$\mathcal{L}_{\text{CE}}(\theta) = -\sum_{i=1}^{C} y_i \ln(\hat{y}_i) = -\ln\left(\hat{y}_{\text{target}}\right) = -\ln\left(\frac{e^{z_{\text{target}}}}{\sum_{j=1}^{C} e^{z_j}}\right). \tag{1.33}$$

Figure 1.8 compares the mathematical profiles of MSE and BCE for a positive target $y = 1$.



(a) MSE Loss for $y = 1$ (Bounded: $\mathcal{L} \in [0,1]$)



(b) BCE Loss for $y = 1$ (Asymptotic Penalty)

Figure 1.8: Comparison of loss function behaviors for target $y = 1$. While MSE is bounded quadratically at 1.0, BCE imposes an asymptotic infinite penalty ($\mathcal{L} \to \infty$) as predicted probability $\hat{y} \to 0$, providing steep, non-vanishing corrective gradients.

1.5.2 Rigorous proof: why MSE fails for classification

A fundamental theoretical question is: *Why do we never use Mean Squared Error with sigmoid activation for classification?* The answer lies in gradient saturation and non-convexity.

Theorem 1.3: *Gradient saturation of MSE under sigmoid activation*

Let a binary classification model predict $\hat{y} = \sigma(z) = \frac{1}{1+e^{-z}}$ with ground-truth target $y \in \{0,1\}$. Under the quadratic loss $\mathcal{L}_{\text{MSE}} = \frac{1}{2}(y-\hat{y})^2$, the gradient with respect to the

pre-activation logit z is:

$$\frac{\partial \mathcal{L}_{\text{MSE}}}{\partial z} = -(y - \hat{y}) \cdot \hat{y}(1 - \hat{y}). \tag{1.34}$$

Conversely, under Binary Cross-Entropy $\mathcal{L}_{\text{BCE}} = -y \ln(\hat{y}) - (1 - y) \ln(1 - \hat{y})$, the gradient is:

$$\frac{\partial \mathcal{L}_{\text{BCE}}}{\partial z} = \hat{y} - y. \tag{1.35}$$

To demonstrate the deficiency of MSE for classification, we analyze the gradients of both loss formulations with respect to the pre-activation logit z.

Applying the calculus chain rule to the Mean Squared Error loss:

$$\frac{\partial \mathcal{L}_{\text{MSE}}}{\partial z} = \frac{\partial \mathcal{L}_{\text{MSE}}}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial z}. \tag{1.36}$$

The derivative of the loss with respect to $\hat{y}$ is:

$$\frac{\partial \mathcal{L}_{\text{MSE}}}{\partial \hat{y}} = \frac{\partial}{\partial \hat{y}} \left[\frac{1}{2}(y - \hat{y})^2 \right] = -(y - \hat{y}).$$

The derivative of the sigmoid is $\sigma'(z) = \sigma(z)(1 - \sigma(z)) = \hat{y}(1 - \hat{y})$. Substituting:

$$\frac{\partial \mathcal{L}_{\text{MSE}}}{\partial z} = -(y - \hat{y})\hat{y}(1 - \hat{y}). \tag{1.37}$$

Next, evaluating the gradient of the Binary Cross-Entropy loss with respect to logit z:

$$\mathcal{L}_{\text{BCE}} = -y \ln(\sigma(z)) - (1 - y) \ln(1 - \sigma(z)). \tag{1.38}$$

Differentiating directly with respect to logit z:

$$\begin{aligned}
\frac{\partial \mathcal{L}_{\text{BCE}}}{\partial z} &= -y \frac{\sigma'(z)}{\sigma(z)} - (1 - y) \frac{-\sigma'(z)}{1 - \sigma(z)} \\
&= -y \frac{\sigma(z)(1 - \sigma(z))}{\sigma(z)} + (1 - y) \frac{\sigma(z)(1 - \sigma(z))}{1 - \sigma(z)} \\
&= -y(1 - \sigma(z)) + (1 - y)\sigma(z) \\
&= -y + y\sigma(z) + \sigma(z) - y\sigma(z) \\
&= \sigma(z) - y = \hat{y} - y. \quad \blacksquare
\end{aligned}$$

The fundamental architectural flaws of MSE for classification are immediately apparent from Equations (1.37) and (1.35):

- **The Worst-Case MSE Pathology (Confident and Wrong)**: Suppose the true label is $y = 1$, but the network is completely mistaken, predicting $z = -10 \implies \hat{y} \approx 0.000045$. The prediction error is maximal: $(y - \hat{y}) \approx 1.0$. However, evaluating the MSE gradient:

$$\frac{\partial \mathcal{L}_{\text{MSE}}}{\partial z} = -(1.0) \times (0.000045) \times (0.999955) \approx -0.000045 \approx 0!$$

  The gradient completely vanishes! Even though the model is entirely wrong, parameter updates halt because the sigmoid derivative $\hat{y}(1 - \hat{y})$ saturates to zero. Optimization gets trapped on a flat plateau.

- **The BCE Remedy**: Under BCE, for the identical scenario ($y = 1, \hat{y} \approx 0$):

$$\frac{\partial \mathcal{L}_{\text{BCE}}}{\partial z} = \hat{y} - y = 0 - 1 = -1.0.$$

The sigmoid derivative in the numerator cancels perfectly with the denominator in the derivative of the logarithm! The gradient is strictly linear with respect to the error $(y - \hat{y})$ and provides maximum learning signal precisely when the model is most wrong.

- **Loss Surface Geometry and Convexity in Logit Space**: As formalized by Goodfellow et al. [**?**], the fundamental geometric pathology of combining MSE with sigmoidal classification units is the destruction of convexity.

To see this analytically, evaluate the second derivative of the loss with respect to pre-activation logit z. For MSE with $\hat{y} = \sigma(z)$:

$$\begin{aligned}\frac{\partial^2 \mathcal{L}_{\text{MSE}}}{\partial z^2} &= \frac{\partial}{\partial z}\left[-(y - \sigma(z))\sigma'(z)\right] \\ &= (\sigma'(z))^2 - (y - \sigma(z))\sigma''(z). \end{aligned} \tag{1.39}$$

Recalling that $\sigma''(z) = \sigma'(z)(1 - 2\sigma(z))$, we have:

$$\frac{\partial^2 \mathcal{L}_{\text{MSE}}}{\partial z^2} = \sigma'(z)\left[\sigma'(z) - (y - \sigma(z))(1 - 2\sigma(z))\right]. \tag{1.40}$$

When the prediction is severely wrong (e.g., $y = 1$ and $z \ll 0$ so $\sigma(z) \to 0$), the term $(y - \sigma(z))(1 - 2\sigma(z)) \approx (1)(1) = 1$, which strictly dominates $\sigma'(z) \approx 0$. Consequently, the bracketed quantity becomes negative, causing the second derivative to flip sign or vanish. This sign reversal breaks positive semi-definiteness, creating vast non-convex plateaus, inflection points, and saddle regions where gradient descent stalls indefinitely.

In stark contrast, for Binary Cross-Entropy, the first derivative is $\frac{\partial \mathcal{L}_{\text{BCE}}}{\partial z} = \sigma(z) - y$. Differentiating again with respect to z:

$$\frac{\partial^2 \mathcal{L}_{\text{BCE}}}{\partial z^2} = \frac{\partial}{\partial z}(\sigma(z) - y) = \sigma'(z) = \sigma(z)(1 - \sigma(z)) > 0 \quad \forall z \in \mathbb{R}. \tag{1.41}$$

Because $\sigma(z) \in (0, 1)$ for all finite $z \in \mathbb{R}$, the second derivative is strictly positive everywhere. The BCE loss surface is therefore **strictly convex** with respect to the logit z. In logit space, there are no local plateaus, no saddle points, and any stationary point is guaranteed to be the unique global minimum.

1.5.3 Maximum likelihood estimation derivation of cross-entropy

Cross-entropy is not an empirical heuristic; it is the mathematically canonical loss derived from Maximum Likelihood Estimation (MLE).

Bernoulli likelihood derivation (BCE)

For binary classification, target $y_i \in \{0, 1\}$ follows a Bernoulli distribution parameterized by probability $p_i = \hat{y}_i = \sigma(z_i)$:

$$\mathbb{P}(Y = y_i \mid \mathbf{x}_i) = \hat{y}_i^{y_i}(1 - \hat{y}_i)^{1-y_i}. \tag{1.42}$$

For an independent dataset of N samples, the total likelihood is $\mathcal{L}_{\text{data}}(\theta) = \prod_{i=1}^{N} \hat{y}_i^{y_i}(1 - \hat{y}_i)^{1-y_i}$. Taking the negative log-likelihood (NLL):

$$\mathcal{L}(\theta) = -\ln \mathcal{L}_{\text{data}}(\theta) = -\sum_{i=1}^{N}\left[y_i \ln \hat{y}_i + (1 - y_i)\ln(1 - \hat{y}_i)\right] = \sum_{i=1}^{N} \mathcal{L}_{\text{BCE},i}. \tag{1.43}$$

Minimizing BCE is mathematically identical to maximizing the data likelihood under a Bernoulli distribution.

Categorical likelihood derivation (CCE)

For multi-class classification over C classes with one-hot targets $\mathbf{y}_i$, the likelihood follows a Categorical distribution:

$$\mathbb{P}(\mathbf{y}_i \mid \mathbf{x}_i) = \prod_{c=1}^{C} \hat{y}_{ic}^{\,y_{ic}} \implies -\ln \mathbb{P}(\mathbf{y}_i \mid \mathbf{x}_i) = -\sum_{c=1}^{C} y_{ic} \ln \hat{y}_{ic} = \mathcal{L}_{\text{CE}}. \tag{1.44}$$

1.5.4 Gradient descent and adaptive optimizers

Parameters θ are updated iteratively along the negative gradient:

$$\theta^{(t+1)} := \theta^{(t)} - \alpha \nabla_\theta \mathcal{L}(\theta^{(t)}), \tag{1.45}$$

where $\alpha > 0$ is the learning rate.

In deep neural architectures, vanilla Gradient Descent struggles with:

1. **Ravines and Ill-Conditioned Curvature**: Valleys where gradient magnitude is large across the ravine walls but tiny along the base, causing erratic cross-valley oscillation.
2. **Saddle Points**: Points where $\nabla \mathcal{L} = 0$ while the Hessian possesses both positive and negative eigenvalues.

- **Momentum**: Incorporates an exponentially decaying velocity vector $\mathbf{v}_t = \beta \mathbf{v}_{t-1} + (1 - \beta)\nabla \mathcal{L}(\theta_t)$, accelerating descent through saddle regions.
- **Adam (Adaptive Moment Estimation) [?]**: Tracks running averages of both gradients (first moment $\mathbf{m}_t$) and squared gradients (second uncentered moment $\mathbf{v}_t$), with bias corrections:

$$\mathbf{m}_t = \beta_1 \mathbf{m}_{t-1} + (1 - \beta_1)\mathbf{g}_t, \quad \hat{\mathbf{m}}_t = \frac{\mathbf{m}_t}{1 - \beta_1^t}, \tag{1.46}$$

$$\mathbf{v}_t = \beta_2 \mathbf{v}_{t-1} + (1 - \beta_2)\mathbf{g}_t^2, \quad \hat{\mathbf{v}}_t = \frac{\mathbf{v}_t}{1 - \beta_2^t}, \tag{1.47}$$

$$\theta_{t+1} = \theta_t - \frac{\alpha}{\sqrt{\hat{\mathbf{v}}_t} + \epsilon} \hat{\mathbf{m}}_t. \tag{1.48}$$

Adam scales learning rates adaptively per parameter, making it the standard optimizer across LLM pre-training and fine-tuning pipelines.

1.6 Computational graphs and exact adjoint backpropagation

To compute gradients efficiently across billions of parameters, deep learning frameworks (PyTorch, JAX) represent mathematical workflows as a *Directed Acyclic Graph (DAG)*, known as a *computational graph*.

Definition 1.2: *Computational graph*

A computational graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ is a directed acyclic graph where:

- Each node $v \in \mathcal{V}$ represents an input variable, parameter, or primitive mathematical operation $(+, \times, -, \exp)$.
- Each directed edge $e = (u, v) \in \mathcal{E}$ represents intermediate data flow from variable u into operation v.

1.6.1 Exact scalar adjoint backpropagation on a computational DAG

To build foundational intuition for how reverse-mode automatic differentiation operates on directed acyclic graphs, consider a canonical scalar computational model.

Consider a scalar computational model with parameters $\theta_1, \theta_2 \in \mathbb{R}$ evaluated on a training pair $(x, y) \in \mathbb{R}^2$:

$$\text{Model Prediction:} \quad \hat{y} = \theta_1 \theta_2 x, \tag{1.49}$$

$$\text{Objective Loss:} \quad \mathcal{L} = (\theta_1 \theta_2 x - y)^2. \tag{1.50}$$

The graph decomposes Equation (1.50) into four primitive operations:

$$a = \theta_1 \theta_2 \quad \text{(parameter interaction product)}, \tag{1.51}$$

$$b = ax = \theta_1 \theta_2 x \quad \text{(prediction } \hat{y}\text{)}, \tag{1.52}$$

$$c = b - y = \theta_1 \theta_2 x - y \quad \text{(residual error)}, \tag{1.53}$$

$$\mathcal{L} = c^2 = (\theta_1 \theta_2 x - y)^2 \quad \text{(scalar squared loss)}. \tag{1.54}$$

Backpropagation computes the derivative of the scalar loss $\mathcal{L}$ with respect to every intermediate variable and parameter by traversing the computational DAG in reverse topological order:

$$\text{Step 1:} \quad \frac{\partial \mathcal{L}}{\partial c} = \frac{\partial (c^2)}{\partial c} = 2c. \tag{1.55}$$

$$\text{Step 2:} \quad \frac{\partial \mathcal{L}}{\partial b} = \frac{\partial \mathcal{L}}{\partial c} \frac{\partial c}{\partial b} = 2c \cdot \frac{\partial (b - y)}{\partial b} = 2c \cdot 1 = 2c. \tag{1.56}$$

$$\text{Step 3:} \quad \frac{\partial \mathcal{L}}{\partial a} = \frac{\partial \mathcal{L}}{\partial b} \frac{\partial b}{\partial a} = 2c \cdot \frac{\partial (ax)}{\partial a} = 2c \cdot x = 2cx. \tag{1.57}$$

$$\text{Step 4:} \quad \frac{\partial \mathcal{L}}{\partial \theta_1} = \frac{\partial \mathcal{L}}{\partial a} \frac{\partial a}{\partial \theta_1} = (2cx) \cdot \frac{\partial (\theta_1 \theta_2)}{\partial \theta_1} = 2cx \cdot \theta_2 = 2(\theta_1 \theta_2 x - y) x \theta_2. \tag{1.58}$$

$$\text{Step 5:} \quad \frac{\partial \mathcal{L}}{\partial \theta_2} = \frac{\partial \mathcal{L}}{\partial a} \frac{\partial a}{\partial \theta_2} = (2cx) \cdot \frac{\partial (\theta_1 \theta_2)}{\partial \theta_2} = 2cx \cdot \theta_1 = 2(\theta_1 \theta_2 x - y) x \theta_1. \tag{1.59}$$

Figure 1.9 illustrates this complete forward and backward adjoint execution.

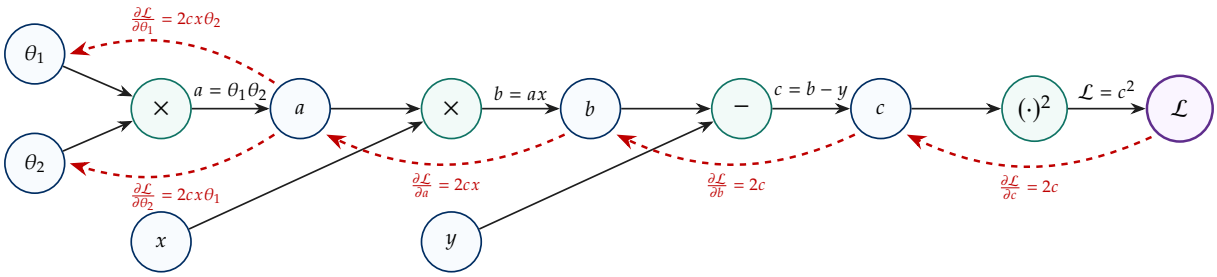


Figure 1.9: Directed Acyclic Computational Graph (DAG) for model $\hat{y} = \theta_1\theta_2 x$ with quadratic loss $\mathcal{L} = (\theta_1\theta_2 x - y)^2$. Solid black arrows trace forward execution; dashed red arrows trace reverse-mode adjoint backpropagation via the chain rule [**?**].

Geometric perturbation perspective

Grant Sanderson (3Blue1Brown) articulates backpropagation not as abstract calculus algebra, but as a systematic tracking of sensitivity nudges. Imagine wiggling parameter θ_1 by an infinitesimal perturbation ϵ. That nudge ripples through the product node to scale a by $\epsilon\theta_2$. The change in a wiggles b by $\epsilon\theta_2 x$, which directly shifts error c and finally scales loss $\mathcal{L}$ by $2c\epsilon\theta_2 x$. Backpropagation simply reads these connected multiplicative sensitivity tracks in reverse, gathering the composite derivative at each fork in the graph.

Mechanics of reverse-mode autodiff engines: Micrograd

Andrej Karpathy's open-source library Micrograd implements reverse-mode automatic differentiation in fewer than 100 lines of pure Python, demonstrating the exact computational graph mechanics shown in Figure 1.9.

Each scalar variable is wrapped inside a `Value` class containing:

- `.data`: the floating-point scalar computed during the forward pass.
- `.grad`: the accumulated adjoint derivative $\partial\mathcal{L}/\partial\cdot$, initialized to 0.0.
- `._prev`: a set storing pointers to operand children nodes.
- `._backward`: a closure storing the local multivariate chain rule.

For example, multiplication is implemented as:

```
def __mul__(self, other):
    other = other if isinstance(other, Value) else Value(other)
    out = Value(self.data * other.data, (self, other), '*')
    def _backward():
        self.grad += other.data * out.grad
        other.grad += self.data * out.grad
    out._backward = _backward
    return out
```

When calling `loss.backward()`, the engine performs a **topological sort** on the DAG to establish reverse dependency ordering, sets `loss.grad = 1.0`, and executes `node._backward()` across all parent nodes in reverse topological order.

1.6.2 Reverse-mode automatic differentiation: Vector-Jacobian products (VJPs)

As formalized by Bishop & Bishop [**?**], the theoretical foundation of backpropagation rests on the duality between *forward-mode* and *reverse-mode* automatic differentiation.

Consider a multi-stage computational sequence where an intermediate vector $\mathbf{u} \in \mathbb{R}^{D_u}$ maps to $\mathbf{v} \in \mathbb{R}^{D_v}$ via differentiable mapping $\mathbf{v} = \mathbf{g}(\mathbf{u})$, eventually yielding a scalar training loss $\mathcal{L} \in \mathbb{R}$. The sensitivity of the mapping is captured by its Jacobian matrix:

$$\mathbf{J_g} = \frac{\partial \mathbf{v}}{\partial \mathbf{u}} \in \mathbb{R}^{D_v \times D_u}, \quad (\mathbf{J_g})_{jk} = \frac{\partial v_j}{\partial u_k}. \tag{1.60}$$

Forward-mode automatic differentiation (Jacobian-Vector Products)

Forward mode propagates directional perturbation vectors $\mathbf{r}_u \in \mathbb{R}^{D_u}$ from inputs toward outputs by evaluating *Jacobian-Vector Products* (JVPs):

$$\mathbf{r}_v = \mathbf{J_g}\mathbf{r}_u \in \mathbb{R}^{D_v}. \tag{1.61}$$

To calculate the complete gradient of loss $\mathcal{L}$ with respect to a parameter vector of dimension P (where $P = D_{\text{in}}$), forward-mode must seed directional vectors along each coordinate basis $\mathbf{e}_1, \ldots, \mathbf{e}_P$ sequentially. Computing the full gradient vector requires P separate forward sweeps, yielding a computational complexity of:

$$\text{Cost}_{\text{forward-mode}}(\nabla_\theta \mathcal{L}) = O(P \times \text{Cost(forward pass)}). \tag{1.62}$$

In modern foundation models where parameter count reaches billions ($P \sim 10^9 - 10^{11}$), forward-mode differentiation is entirely infeasible.

Reverse-mode automatic differentiation (Vector-Jacobian Products)

Reverse mode (backpropagation) inverts the direction of differentiation. Beginning at the final scalar loss $\mathcal{L} \in \mathbb{R}$ with initial adjoint seed $\bar{\mathcal{L}} = \frac{\partial \mathcal{L}}{\partial \mathcal{L}} = 1.0$, the algorithm propagates adjoint row vectors $\bar{\mathbf{v}} = \left(\frac{\partial \mathcal{L}}{\partial \mathbf{v}}\right)^\top \in \mathbb{R}^{1 \times D_v}$ backward through the DAG by evaluating *Vector-Jacobian Products* (VJPs):

$$\bar{\mathbf{u}} = \bar{\mathbf{v}}\mathbf{J_g} = \left(\frac{\partial \mathcal{L}}{\partial \mathbf{v}}\right)^\top \frac{\partial \mathbf{v}}{\partial \mathbf{u}} \in \mathbb{R}^{1 \times D_u}. \tag{1.63}$$

Because the objective $\mathcal{L}$ is a single scalar ($D_{\text{out}} = 1$), a **single reverse traversal** of the computational graph simultaneously evaluates the exact partial derivatives with respect to all P parameters. The computational cost of reverse-mode autodiff scales independently of parameter count P:

$$\text{Cost}_{\text{reverse-mode}}(\nabla_\theta \mathcal{L}) \leq \omega \cdot \text{Cost(forward pass)}, \quad \omega \in [2, 5]. \tag{1.64}$$

Crucially, production deep learning runtimes never materialize the full dense Jacobian matrix $\mathbf{J_g}$ in GPU memory (which would require $O(D_u D_v)$ memory). Instead, each primitive node provides an analytic VJP operator that computes the vector-matrix product directly. For linear projections, this operator computes the matrix-vector products $\mathbf{W}\boldsymbol{\delta}$ and outer products $\mathbf{x}\boldsymbol{\delta}^\top$ derived below.

1.6.3 Vectorized matrix adjoint backpropagation for dense layers

While scalar computational graphs illustrate the chain rule clearly, production deep learning libraries execute backpropagation via *vectorized matrix calculus* across high-dimensional tensors. We now derive the exact vectorized matrix adjoint equations for fully connected layers with explicit tensor dimensions.

Consider a fully connected dense layer mapping an input vector $\mathbf{x} \in \mathbb{R}^{d_{\text{in}}}$ through weight matrix $\mathbf{W} \in \mathbb{R}^{d_{\text{in}} \times d_{\text{out}}}$ and bias $\mathbf{b} \in \mathbb{R}^{d_{\text{out}}}$ to pre-activation $\mathbf{z} \in \mathbb{R}^{d_{\text{out}}}$ and activation $\mathbf{y} \in \mathbb{R}^{d_{\text{out}}}$:

$$\mathbf{z} = \mathbf{W}^\top \mathbf{x} + \mathbf{b} \in \mathbb{R}^{d_{\text{out}}}, \tag{1.65}$$

$$\mathbf{y} = f(\mathbf{z}) \in \mathbb{R}^{d_{\text{out}}}, \tag{1.66}$$

where scalar component z_j expands as:

$$z_j = \sum_{i=1}^{d_{\text{in}}} W_{ij} x_i + b_j. \tag{1.67}$$

The layer output $\mathbf{y}$ propagates downstream to contribute to a final scalar objective loss $\mathcal{L} \in \mathbb{R}$.

Assume downstream backpropagation has computed the loss gradient with respect to layer pre-activation $\mathbf{z}$. We define the adjoint error vector $\boldsymbol{\delta} \in \mathbb{R}^{d_{\text{out}}}$:

$$\boldsymbol{\delta} \triangleq \frac{\partial \mathcal{L}}{\partial \mathbf{z}} = \frac{\partial \mathcal{L}}{\partial \mathbf{y}} \odot f'(\mathbf{z}) \in \mathbb{R}^{d_{\text{out}}}, \tag{1.68}$$

where $\odot$ denotes the elementwise Hadamard product.

Weight matrix adjoint derivation

To compute the derivative of scalar loss $\mathcal{L}$ with respect to individual matrix element W_{ij}, we apply the multivariate chain rule across all output pre-activations z_k:

$$\frac{\partial \mathcal{L}}{\partial W_{ij}} = \sum_{k=1}^{d_{\text{out}}} \frac{\partial \mathcal{L}}{\partial z_k} \frac{\partial z_k}{\partial W_{ij}}. \tag{1.69}$$

Because $z_k = \sum_{l=1}^{d_{\text{in}}} W_{lk} x_l + b_k$, the partial derivative is non-zero only when $k = j$:

$$\frac{\partial z_k}{\partial W_{ij}} = \begin{cases} x_i & \text{if } k = j \\ 0 & \text{if } k \neq j \end{cases}. \tag{1.70}$$

Substituting this back:

$$\frac{\partial \mathcal{L}}{\partial W_{ij}} = \frac{\partial \mathcal{L}}{\partial z_j} x_i = \delta_j x_i = x_i \delta_j. \tag{1.71}$$

Collecting all elements W_{ij} into matrix form reveals that the weight gradient is identically the **outer product** of the upstream input vector $\mathbf{x} \in \mathbb{R}^{d_{\text{in}}}$ and the adjoint error vector $\boldsymbol{\delta} \in \mathbb{R}^{d_{\text{out}}}$:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}} = \mathbf{x} \boldsymbol{\delta}^\top \in \mathbb{R}^{d_{\text{in}} \times d_{\text{out}}}. \tag{1.72}$$

Bias vector adjoint derivation

For each scalar bias component b_j:

$$\frac{\partial \mathcal{L}}{\partial b_j} = \sum_{k=1}^{d_{\text{out}}} \frac{\partial \mathcal{L}}{\partial z_k} \frac{\partial z_k}{\partial b_j} = \delta_j \cdot 1 = \delta_j. \tag{1.73}$$

In vector notation:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{b}} = \boldsymbol{\delta} \in \mathbb{R}^{d_{\text{out}}}. \tag{1.74}$$

Upstream input adjoint derivation

To propagate error signals back to previous layers in a deep neural network, we must evaluate the gradient of loss $\mathcal{L}$ with respect to the input feature vector $\mathbf{x} \in \mathbb{R}^{d_{\text{in}}}$. Applying the chain rule to input component x_i:

$$\frac{\partial \mathcal{L}}{\partial x_i} = \sum_{k=1}^{d_{\text{out}}} \frac{\partial \mathcal{L}}{\partial z_k} \frac{\partial z_k}{\partial x_i} = \sum_{k=1}^{d_{\text{out}}} \delta_k W_{ik} = \sum_{k=1}^{d_{\text{out}}} W_{ik} \delta_k. \tag{1.75}$$

Recognizing this sum as the i-th element of the matrix-vector multiplication between weight matrix $\mathbf{W} \in \mathbb{R}^{d_{\text{in}} \times d_{\text{out}}}$ and adjoint vector $\boldsymbol{\delta} \in \mathbb{R}^{d_{\text{out}}}$:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{x}} = \mathbf{W}\boldsymbol{\delta} \in \mathbb{R}^{d_{\text{in}}}. \tag{1.76}$$

Figure 1.10 diagrams this vectorized adjoint error propagation across the dense layer.

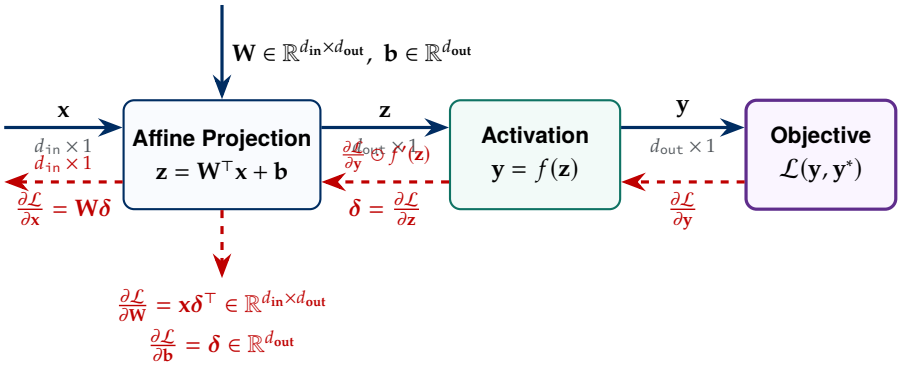


Figure 1.10: Vectorized matrix adjoint backpropagation across a fully connected dense layer. Solid navy arrows trace forward execution mapping input $\mathbf{x} \in \mathbb{R}^{d_{\text{in}}}$ to output $\mathbf{y} \in \mathbb{R}^{d_{\text{out}}}$; dashed red arrows trace reverse-mode adjoint error propagation ($\boldsymbol{\delta} \in \mathbb{R}^{d_{\text{out}}}$), yielding parameter updates $\frac{\partial \mathcal{L}}{\partial \mathbf{W}} = \mathbf{x}\boldsymbol{\delta}^\top$, $\frac{\partial \mathcal{L}}{\partial \mathbf{b}} = \boldsymbol{\delta}$, and upstream gradient $\frac{\partial \mathcal{L}}{\partial \mathbf{x}} = \mathbf{W}\boldsymbol{\delta}$.

Mini-batch matrix generalization

In GPU implementations, B samples are grouped into a batch data matrix $\mathbf{X} \in \mathbb{R}^{B \times d_{\text{in}}}$. The layer computes:

$$\mathbf{Z} = \mathbf{X}\mathbf{W} + \mathbf{1}_B \mathbf{b}^\top \in \mathbb{R}^{B \times d_{\text{out}}}, \quad \mathbf{Y} = f(\mathbf{Z}) \in \mathbb{R}^{B \times d_{\text{out}}}. \tag{1.77}$$

Defining batch adjoint matrix $\boldsymbol{\Delta} = \frac{\partial \mathcal{L}}{\partial \mathbf{Z}} \in \mathbb{R}^{B \times d_{\text{out}}}$, the gradient equations generalize to highly efficient Basic Linear Algebra Subprograms (BLAS) matrix multiplications:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}} = \mathbf{X}^\top \boldsymbol{\Delta} \in \mathbb{R}^{d_{\text{in}} \times d_{\text{out}}}, \tag{1.78}$$

$$\frac{\partial \mathcal{L}}{\partial \mathbf{b}} = \boldsymbol{\Delta}^\top \mathbf{1}_B \in \mathbb{R}^{d_{\text{out}}}, \tag{1.79}$$

$$\frac{\partial \mathcal{L}}{\partial \mathbf{X}} = \boldsymbol{\Delta}\mathbf{W}^\top \in \mathbb{R}^{B \times d_{\text{in}}}. \tag{1.80}$$

These matrix equations form the computational workhorse executed across billions of parameters in modern GPU training clusters.

1.7 The sequential computation impasse: bridge to recurrent neural networks

In this chapter, we have formulated the mathematical and architectural foundations of deep feedforward neural networks (FCNNs / Multi-Layer Perceptrons):

1. **Continuous Units**: The artificial perceptron computes weighted inner products over real-valued vectors.
2. **Non-Linearity**: Non-linear activations (σ, tanh, ReLU, GELU) prevent multi-layer linear stacking collapse, folding space to solve linearly inseparable problems like XOR and enabling universal function approximation.
3. **Principled Optimization**: Cross-entropy objective functions provide linear error signals without gradient saturation, and backpropagation calculates matrix adjoints across computational DAGs via vectorized matrix calculus.

Throughout these formulations, feedforward networks operate under a fundamental structural constraint: the input is a static, fixed-dimensional Euclidean vector $\mathbf{x} \in \mathbb{R}^{d_{\text{in}}}$, and the network maps this vector through a cascade of static linear projections $\mathbf{W}_l \in \mathbb{R}^{d_l \times d_{l-1}}$ to produce a fixed-dimensional output $\mathbf{y} \in \mathbb{R}^{d_{\text{out}}}$:

$$\mathbf{y} = \mathcal{F}(\mathbf{x}; \Theta) = f_L(\mathbf{W}_L \cdots f_1(\mathbf{W}_1\mathbf{x} + \mathbf{b}_1) \cdots + \mathbf{b}_L). \tag{1.81}$$

While this architecture is highly effective for tabular data and fixed-attribute classification, it immediately encounters a computational impasse when confronted with sequential data—such as human natural language, software source code tokens, execution traces, and temporal signals. Sequential data violates the core assumptions of feedforward processing in three fundamental ways:

1. **Rigid Input Dimensionality**: Natural language sentences and source code blocks have elastic, variable lengths T. An FCNN strictly expects an input of dimension d_{in}, forcing artificial truncation or massive zero-padding that wastes computational throughput and memory.
2. **Zero Internal State and Temporal Memory**: Feedforward networks are memoryless. Each forward pass $\mathcal{F}(\mathbf{x})$ is independent and decoupled from prior inputs. They cannot maintain temporal context or resolve long-range grammatical and syntactic dependencies across multiple time steps.
3. **Absence of Parameter Sharing Across Positions**: In an FCNN, each position in the input is governed by separate weight parameters. If a syntactic construct (such as a variable declaration `"let x = 5;"` or a noun phrase *"a large language model"*) appears at token position 1 versus token position 20, the FCNN must learn separate weight matrices for each position independently, destroying sample efficiency.

To accommodate variable-length sequences, maintain dynamic temporal context, and achieve position-invariant parameter efficiency, deep learning requires a recurrence mechanism with shared weights across time.

This architectural necessity leads directly to Chapter **??**: *Recurrent Neural Networks and Sequence Modeling*.